A Provenance-Aware Core Calculus for Triton-Distributed Synchronization

Baber Rehman
Huawei Technologies, Hong Kong

Abstract

Triton-Distributed exposes signal-based synchronization operations intended to order communication and computation in distributed accelerator kernels. Reasoning about the intended wait–token–load discipline is difficult because the Python API, compiler IR, and backend memory model provide different parts of the contract. We present a mechanized core calculus that isolates one such discipline and makes its obligations explicit. The Lean 4 model includes typed values and environments, notification state, successful waits, token consumption, and guarded loads. Its transition semantics associates pending and consumed capabilities with signal, rank, pointer, scope, and ordering metadata. We prove preservation of capability origins across traces and a universal provenance theorem: from an initial state with no pending or consumed capabilities, every permitted load has an earlier matching wait and consume in the abstract action trace. We also mechanize a three-step producer–consumer trace and negative examples that exclude loads without the required abstract capabilities. A source-pattern validator checks three representative upstream tutorial shapes against a pinned Triton-distributed checkout. These results establish properties of the encoded calculus and checked source patterns only. They do not establish a source-to-core simulation, compiler correctness, backend memory-model refinement, or hardware safety. We describe the abstraction boundary and identify the additional translation, compiler, and target evidence needed to support stronger claims.

2012 ACM Subject Classification Software and its engineering → Formal software verification; Software and its engineering → Concurrency

Keywords and phrases Lean, distributed accelerators, synchronization, provenance, operational semantics

Digital Object Identifier 10.4230/LIPIcs...

Supplementary Material Lean formalization and validation sources in the accompanying artifact

1 Introduction

Distributed accelerators make communication part of the computation: while one rank produces data, another may need to wait for a signal, establish a dependency on the data, and only then load it. Getting this order wrong can leave a kernel with values that are stale or otherwise not justified by the intended communication protocol. Yet the relevant contract is distributed across layers. An API can describe the shape of `notify`, `wait`, and `consume_token`; an intermediate representation can encode dependencies; compiler transformations can preserve or alter those dependencies; and a backend must map the operations to target-specific ordering and visibility. A source-level sequence that looks right is not, by itself, evidence that all these layers agree.

This creates a verification gap with a deceptively simple symptom: an ordinary load can be well typed and still lack a valid synchronization history. Scalar and pointer types tell us what a value represents, but not whether the consumer has waited for the matching signal, whether the resulting token was consumed for this data value, or whether the load is covered by that consumption. Conversely, proving that a particular example follows the expected sequence does not establish that every load admitted by a semantics has the required history. A useful first step is therefore to make the protocol obligation precise at the level where it can be stated and checked: the transition trace.


© Baber Rehman;
licensed under Creative Commons License CC-BY 4.0


For example, a consumer might wait on signal s_A but associate the resulting token with data pointer p_B:

```
token = wait(signal_A, ...)
data = consume_token(data_ptr_B, token)
value = tl.load(data)
```

The API accepts the token and value as separate operands; it does not generally check that the signal waited on protects the data addressed by `data_ptr_B`. The formal transition relation rejects this mismatched protocol: its pending capability records the waited-on signal and rank, and consumption and loading must match the relevant pointer and capability metadata. This is an illustrative model-to-interface gap, not a report of an observed upstream kernel failure; whether a particular mismatch produces incorrect machine behavior depends on the source program and compiler/backend execution.

We study that obligation in a small Lean 4 core calculus motivated by Triton-Distributed's documented producer–consumer idiom. The calculus separates expression typing and evaluation from machine transitions, and represents synchronization evidence explicitly. A successful wait creates a pending capability carrying signal, rank, expected value, scope, and ordering metadata; consuming a token for a data value turns that evidence into a consumed record for the value variable, token variable, signal address, and rank. A load is admitted when that record matches its pointer variable, rank, and address. Crucially, notification visibility is an explicit premise of successful waiting. The model does not silently derive a hardware fact that it has not formalized.

The central result is a universal provenance theorem, not merely a checked execution: for every trace admitted by the transition relation from an initial machine with no pending or consumed capabilities, any permitted load has an earlier wait followed by a consume using the same token variable. The consumed record must match the loaded pointer variable, rank, and address; the consume must match a pending record's token variable, rank, address, scope, and ordering. However, the consumed record omits the expected value, scope, and ordering, so these are not matched at the final load. The theorem rules out a class of internally inconsistent abstract executions and makes the required evidence inspectable in Lean. This gives a precise local invariant that can guide stronger verification: a future source-to-core translation could show that source dependencies create the modeled capabilities, and a backend refinement could show that generated operations preserve their ordering and visibility.

That boundary is essential to the contribution. We do not prove that arbitrary Triton-Distributed programs translate into this calculus, that compiler passes preserve its protocol, or that any hardware backend satisfies the visibility premise. Nor do we claim deadlock freedom, general race freedom, or end-to-end kernel safety. We instead formalize one synchronization discipline, prove what follows from it, audit selected tutorial-shaped source patterns, and identify the missing links needed to connect the abstract result to executable kernels. This deliberately layered result turns an informal ordering expectation into a machine-checked obligation without confusing that obligation with its still-unproved implementation.

Contributions.

- We define a typed Lean core with notification, wait, consume, and load transitions, and state the notification-visibility condition as an explicit semantic premise.
- We prove that pending and consumed capabilities retain trace origins, and derive a universal wait–consume provenance decomposition for permitted loads from empty capability state.

- We provide executable positive and negative protocol examples, together with a standard-library Python AST validator for selected upstream tutorial call shapes.
- We document the source correspondence and trusted boundary, including the missing source translation, dependency-preservation proof, and backend memory-model refinement.

These contributions should be read as a source-audited formal prototype, not as verification of Triton-Distributed or a claim of priority. Our related-work discussion is focused rather than systematic, and the results do not establish novelty relative to all prior work.

Roadmap and notation. Section 2 records the source contract and abstraction boundary. Section 3 defines the Lean vocabulary, expression evaluator, and transition rules; subsections follow the order of the declarations in `Core.lean` and `Transition.lean`. Section 4 states the mechanized invariants and gives proof decompositions. Section 5 presents concrete traces and source-pattern validation. Section 6 compares the calculus with the implementation, while Section 7 lists assumptions and limitations. We write $x \in L$ for list membership, $L_1 ++ L_2$ for list concatenation, and $m \xrightarrow{a} m'$ for an enabled transition labelled by action a. Lean identifiers are set in typewriter font when the exact declaration is relevant.

2 Background and Source Contract

Distributed kernels execute across ranks that own distinct portions of a computation but may communicate through remotely addressable, symmetric allocations. A rank can form an address for a peer's corresponding allocation, issue communication or computation, and coordinate progress with signals. This setting makes synchronization more than a control-flow concern: the consumer must establish that the producer's relevant data is ready and that the dependency remains attached to the later data access. The paper focuses on one documented notify–wait–consume–load pattern rather than attempting to model this broader execution environment.

The source interface. The audited Triton-Distributed checkout provides rank and world-size queries, symmetric addressing, signal notification, waiting, token consumption, and ordinary data operations. We use the Python API, distributed operation definitions, language and semantics documentation, and the notify–wait tutorial at revision `1e6d688` (2026-08-24). The revision is also recorded in the data-availability statement. These sources establish the operation shapes and motivate the selected pattern; they are not treated as a formal semantics for all source programs.

Producer–consumer pattern. In the tutorial-shaped idiom, a producer writes data and notifies a signal associated with a consumer. The consumer waits for the expected signal state, receives a token, associates that token with a data value using `consume_token(value, token)`, and subsequently loads through the relevant pointer. Informally, the signal communicates progress, while token consumption makes the synchronization dependency relevant to the data use. The exact details of signal setup, allocation, rank selection, and reuse belong to the surrounding program and are outside the small protocol modeled here.

What the token means here. At the source interface, the wait result is represented as an integer token, and `consume_token` is described as introducing a dependency associated with its value operand. An integer type alone cannot express where a token came from or which

data access it justifies. The core therefore distinguishes the token's static scalar type from its abstract metadata and capability history: a successful wait creates pending evidence, a matching consume transforms that evidence into an ordering witness for a value, and a guarded load consults that witness. This is a formal design choice for the calculus, not a claim that the source API's integer token intrinsically carries all of this metadata.

From source operations to abstract actions. The source audit records a deliberately partial correspondence. Rank and symmetric-addressing operations motivate typed rank and pointer values; notify and wait motivate signal and wait actions; consume motivates a transition that requires a matching pending token; and load motivates a transition guarded by an ordering witness. The model abstracts away pointer lowering, address calculation, general tensor values, program control flow, concurrency scheduling, and the complete semantics of the source language. The accompanying AST validator checks selected tutorial call shapes; it is a source-pattern check, not a translation into Lean and not a proof that the source executes according to the core semantics.

Ordering and visibility boundary. The intended reasoning depends on two distinct obligations. First, a successful wait must be justified by a visible notification under the relevant scope and ordering. Second, the dependency introduced by token consumption must remain effective until the data load. In this calculus, the first obligation appears as the explicit premise `NotificationVisible`; the second appears as the requirement for a matching consumed ordering witness at load. Scope and ordering metadata are recorded and matched abstractly, but the model contains no weak-memory execution relation and derives neither obligation from a particular backend. Establishing those facts requires target- and compiler-specific evidence, as detailed in the limitations and memory-model discussion.

The resulting contract is intentionally conditional: if the abstract transition system admits a load, then its state must contain the corresponding protocol evidence. The theorem proved later reconstructs the wait and consume origins of that evidence from the trace. Connecting this conditional result to Triton-Distributed executions would additionally require a source/IR translation, a simulation argument, compiler dependency-preservation evidence, and a backend refinement for notification visibility. The current artifact supplies none of those end-to-end links; it makes their exact role explicit.

3 Core Calculus

This section gives the model in enough detail to reconstruct its definitions and to understand exactly what the proved theorems quantify over. The development is split into two Lean modules. `Core.lean` defines expressions, values, states, typing, and big-step evaluation. `Transition.lean` extends this vocabulary with explicit state transitions, capabilities, traces, and provenance theorems. `Examples.lean` constructs concrete traces and counterexamples over the transition semantics.

The model has three design goals. First, symmetric addressing and loads should preserve pointer element types. Second, successful synchronization should leave an explicit capability that can be matched against a later data access. Third, that capability should have a trace origin, so an unrelated integer token cannot justify a load. Variables, ranks, and addresses are natural numbers; the environment is a function from variables to optional values. This keeps the artifact focused on the protocol rather than on parser, substitution, or thread-scheduling machinery.

The model separates expression evaluation from protocol transitions. The expression relation describes the result of one operation in a fixed state; the transition relation evolves signals and capability state across actions. This separation permits ordinary type preservation to be stated independently from token provenance: a token can have the scalar token type without yet having a valid wait origin.

3.1 Machine state

A machine contains a state, a typed environment, a list of ordering witnesses, a list of pending tokens, and a list of consumed orderings. Values include typed symmetric pointers and tokens carrying address, observed value, scope, and ordering metadata. A pending token records its token variable, signal rank and address, expected signal value, scope, and ordering. A consumed ordering records the value and token variables, signal address, and rank.

3.1.1 Primitive domains

The model defines four data types, represented by the Lean inductive types `DType`, `Scope`, `Ordering`, and `Result`. The data types are `i32`, `i64`, `f16`, and `f32`. A type is a pair $\langle d, n\rangle$, where d is a data type and n is a size. Scopes are `gpu` and `system`; orderings are `relaxed`, `acquire`, `release`, and `acqRel`. Ranks, variables, and addresses are aliases for `Nat`.

The explicit scope and ordering fields are retained even though the current provenance theorem does not model a weak-memory execution. A wait records these fields in its pending capability, and consume requires them to match. They are parameters of the future backend-refinement boundary: a target proof must explain what each pair means on the implementation platform.

3.1.2 Expressions and values

The expression grammar is the following rendering of the `Expr` inductive type:

$$\begin{aligned} e \;::=\;& x \mid \mathsf{rank}(a) \mid \mathsf{numRanks}(a) \mid \mathsf{symmAt}(e, r) \\ \mid\;& \mathsf{notify}(e, e, e, b, s) \mid \mathsf{wait}(e, e, e, s, o) \\ \mid\;& \mathsf{consumeToken}(e, e) \mid \mathsf{load}(e) \mid \mathsf{store}(e, e). \end{aligned}$$

Here a is an integer axis parameter, r is a target rank, b is the Boolean choice between set and add notification, and s, o are scope and ordering. The grammar retains the source operation boundaries. In particular, `consumeToken` is not reduced to an ordinary identity expression, even though it returns its value operand; its protocol effect is represented in the transition layer.

Runtime values include tiles, scalar integers, pointers, tokens, and unit:

$$\begin{aligned} v \;::=\;& \mathsf{tile}(\tau, \vec{z}) \mid \mathsf{i32}(z) \mid \mathsf{i64}(z) \\ \mid\;& \mathsf{ptr}(\tau, r, a, \mathsf{sym}) \mid \mathsf{token}(a, n, s, o) \mid \mathsf{unit}. \end{aligned}$$

The pointer stores an element type, owner rank, address, and a symmetric bit. The token stores signal address, observed value, scope, and ordering. Its static type is still scalar i32; metadata is used by the abstract protocol. The environment ρ maps variables to optional values.

The state is $\sigma = (W, R, M, T, S, O)$, containing world size, current rank, memory, cell types, signals, and signal ownership. Memory maps a rank and address to an optional value;

cell types map the same pair to an optional type. Signals map rank and address to natural counters. The separate memory and cell-type maps support a store well-typedness predicate without introducing a full typed heap semantics.

3.2 Static typing

The context Γ contains a world size and a variable-to-type map. The judgment $\Gamma \vdash e : \tau$, implemented by `HasType`, has one constructor for each expression form. Variables use the type supplied by the context. Rank and world-size queries return scalar $\mathsf{i32}$, while symmetric addressing preserves the operand type and requires a target below the context world size.

The notification constructor requires an $\mathsf{i64}$ scalar signal pointer, an $\mathsf{i32}$ rank, and an $\mathsf{i64}$ signal value; it returns unit. Wait has the same pointer, count, and expected-value shapes and returns an $\mathsf{i32}$ token. Consume requires an arbitrary value type and an $\mathsf{i32}$ token, returning the first operand's type. Loads preserve pointer type and stores return unit.

Typing is intentionally permissive about provenance. A token is well typed because it has the token type; typing alone does not say that it came from a successful wait. The transition-level pending and consumed capabilities provide that stronger fact. Thus the main result is a protocol refinement of the ordinary type system, not a claim that the token type itself encodes lineage.

3.2.1 Typing rules

For reference, the principal typing rules can be displayed compactly as follows. The variable rule reads the optional binding from the context:

$$\frac{\Gamma.vars(x) = \mathsf{some}\ \tau}{\Gamma \vdash x : \tau} \qquad \frac{}{\Gamma \vdash \mathsf{rank}(a) : \mathsf{i32}_1} \qquad \frac{}{\Gamma \vdash \mathsf{numRanks}(a) : \mathsf{i32}_1}.$$

Symmetric addressing preserves the operand type while checking the target:

$$\frac{\Gamma \vdash e : \tau \qquad r < \Gamma.world}{\Gamma \vdash \mathsf{symmAt}(e, r) : \tau}.$$

Writing $\mathsf{i32}_1$ and $\mathsf{i64}_1$ for scalar types and $\mathsf{i32}_0$ for unit, the synchronization rules are:

$$\frac{\Gamma \vdash p : \mathsf{i64}_1 \quad \Gamma \vdash r : \mathsf{i32}_1 \quad \Gamma \vdash v : \mathsf{i64}_1}{\Gamma \vdash \mathsf{notify}(p, r, v, b, s) : \mathsf{i32}_0}$$

and

$$\frac{\Gamma \vdash p : \mathsf{i64}_1 \quad \Gamma \vdash n : \mathsf{i32}_1 \quad \Gamma \vdash v : \mathsf{i64}_1}{\Gamma \vdash \mathsf{wait}(p, n, v, s, o) : \mathsf{i32}_1}.$$

The consume rule is polymorphic in the value type but fixes the token type. The modeled `Ty` contains only dtype and size; it does not distinguish pointers from scalars, so this typing rule alone does not establish that its value operand is a pointer:

$$\frac{\Gamma \vdash v : \tau \qquad \Gamma \vdash t : \mathsf{i32}_1}{\Gamma \vdash \mathsf{consumeToken}(v, t) : \tau}.$$

The load rule preserves its operand type, and the store rule requires matching operand types and returns unit:

$$\frac{\Gamma \vdash p : \tau}{\Gamma \vdash \mathsf{load}(p) : \tau} \qquad \frac{\Gamma \vdash p : \tau \quad \Gamma \vdash v : \tau}{\Gamma \vdash \mathsf{store}(p, v) : \mathsf{i32}_0}.$$

These rules are a direct mathematical rendering of the constructors of `HasType`; they are not an independently implemented type checker.

3.2.2 Runtime typing judgments

Static typing is paired with three runtime predicates. First, `ValueHasType W v` τ relates a value to a type under a world size. For a pointer it requires the stored element type to equal τ and the owner rank to be below the world size. A token is typed as $\mathsf{i32}_1$, so its protocol metadata is not part of ordinary type identity. Second, `EnvHasType` Γ ρ is pointwise:

$$\frac{\Gamma.vars(x) = \mathsf{some}\ \tau \quad \rho(x) = \mathsf{some}\ v}{\mathsf{EnvHasType}(\Gamma, \rho)\ \text{requires}\ \mathsf{ValueHasType}(\Gamma.world, v, \tau)}.$$

Third, `StoreWellTyped` σ quantifies over every declared and present cell and requires its value to have the declared type. Together these predicates state the runtime portion of type safety without requiring every variable to be initialized.

3.3 Value and store typing

The predicate `ValueHasType W v` τ is defined by cases on the value. A tile has the declared type and a payload whose list length equals the type size. Integer values have exactly their scalar types. A pointer has the declared element type and an owner below W. A token has scalar i32 type regardless of metadata, and unit has the zero-sized i32 type.

The environment predicate is pointwise: whenever Γ assigns a type to a variable and the environment assigns a value, that value satisfies `ValueHasType`. The store predicate states that every in-range cell with a declared type and present memory value contains a value of that type. This makes load preservation direct: the load rule supplies the cell lookup, while `StoreWellTyped` supplies value typing.

3.4 Expression evaluation

The relation $\sigma, \rho \vdash e \Downarrow r$, implemented by `Eval`, returns either $\mathsf{value}(v)$ or blocked. Variable lookup returns the environment value. A rank query returns the current rank when it is in range, and a world-size query returns the state world. Symmetric addressing takes a symmetric pointer and a valid target and returns a pointer with the target rank while preserving element type and address.

Notification evaluation checks for a symmetric i64 signal pointer and an in-range rank, returning unit; its signal update is represented by the transition semantics. A ready wait checks a pointer, positive count, expected value, and `NotificationVisible`; it returns a token carrying signal address, expected value, scope, and ordering. A blocked wait checks a positive count and a signal below the expected value and returns `blocked`. Rank-query axis arguments are ignored. Expression-level consume only looks up the value and token operands and returns the value; it does not compare token metadata. The expression rule does not require that the token originated from a wait; that stronger condition belongs to `Step.consume`. Load checks a symmetric pointer, valid rank, matching cell type, and present memory value.

3.5 Expression-level metatheory

The core proves evaluation determinism:

$$\frac{\sigma, \rho \vdash e \Downarrow r_1 \qquad \sigma, \rho \vdash e \Downarrow r_2}{r_1 = r_2}.$$

The Lean theorem `eval_deterministic` proves this by inversion on the two evaluation derivations. `eval_var_preservation` proves that a well-typed variable lookup returns a value of its

context type. Rank, wait, and symmetric-addressing preservation establish the corresponding scalar, token, and pointer results. `eval_load_preservation` uses `StoreWellTyped` to show that a successful load returns a value with the declared cell type.

These are local theorems for a fixed state and environment. They do not imply determinism of a distributed execution, and they do not establish the capability invariant. That invariant is proved over the transition trace.

3.6 Type-safety theorems

The artifact's type-safety results are preservation-style theorems for the encoded language and machine. They should be distinguished from the provenance theorem: type safety establishes that values have the declared shapes, whereas provenance establishes that a load has a synchronization history.

Expression safety. The theorem `eval_var_preservation` states that if $\Gamma \vdash \mathsf{var}(x) : \tau$, the environment is well typed, the context and state have equal world sizes, and evaluation returns v, then

$$\mathsf{ValueHasType}(\sigma.world, v, \tau).$$

This is the fundamental environment lookup lemma. The theorem `eval_rank_preservation` specializes it to rank queries and proves that the returned current rank has type $\mathsf{i32}_1$. The theorem `eval_wait_preservation` proves that every successful wait result has type $\mathsf{i32}_1$, the static token type. Importantly, it does not claim that the token has a valid pending origin.

Theorem `eval_symmAt_safe` states that a successful symmetric addressing evaluation returns a pointer whose element type is the input type and whose target rank is within the state world. Its proof inverts the `Eval.symmAt` derivation and combines pointer typing with the target bound. The theorem `eval_load_preservation` uses a `StoreWellTyped` premise: if a load evaluates successfully, then there exist a cell type, rank, and address such that the returned value has that cell type. This existential conclusion matches the current evaluator, whose load rule obtains the cell type from the state rather than carrying it in the result type.

Machine safety. The transition module packages runtime safety as

$$\mathsf{WellTypedMachine}(\Gamma, m) \triangleq \begin{array}{l} \Gamma.world = m.state.world \wedge \\ \mathsf{EnvHasType}(\Gamma, m.env) \wedge \\ \mathsf{StoreWellTyped}(m.state). \end{array}$$

The theorem `step_preserves_well_typed` proves

$$\mathsf{WellTypedMachine}(\Gamma, m) \wedge \mathsf{Step}(m, a, m') \\ \Longrightarrow \mathsf{WellTypedMachine}(\Gamma, m').$$

The proof is by cases on the step constructor. Notifications change only the signal function, and the point-update definitions preserve the world and all memory cells. Wait, consume, and load leave the state, environment, and cell typing unchanged. The theorem therefore exposes a useful limitation: the current transition system has no data-changing store step, so it proves preservation rather than a full subject-reduction theorem for mutable programs.

Induction over `Trace` lifts the step result to `trace_preserves_well_typed`:

$$\mathsf{WellTypedMachine}(\Gamma, m_1) \wedge \mathsf{Trace}(m_1, A, m_2) \Longrightarrow \mathsf{WellTypedMachine}(\Gamma, m_2).$$

The separate theorem `trace_preserves_world` proves that the state world remains equal to the context world along the same trace. These are the paper's explicit type-safety theorems. They establish preservation of the formal runtime invariants under every admitted transition, not safety of arbitrary source programs or generated accelerator code.

Safety boundary. The type system rejects malformed expression shapes such as a wait whose signal operand is not an $\mathsf{i64}$ scalar pointer or a consume whose token operand is not $\mathsf{i32}_1$. It does not reject every protocol mistake: an integer-typed token with fabricated metadata remains a well-typed value. Such a value is rejected by the transition consume premise when no matching pending capability exists. This division of labor is central to the formalization: static typing protects representation invariants, while the capability trace protects synchronization provenance.

Signal updates are state-changing transitions. A set notification assigns a signal value; an add notification increments it. A wait transition is enabled when the expected value is no greater than the current signal value and the explicit predicate `NotificationVisible` holds. A successful wait adds a witness and a pending-token capability. The predicate is a premise: it is not derived from a hardware memory model.

A consume transition requires a symmetric pointer and token in the environment, matching token and pointer addresses, and a pending capability whose token variable, signal rank, signal address, scope, and ordering match. It does not compare the token's observed value with the pending capability's expected value. It removes pending capabilities for that token variable and records a consumed ordering. A load transition is permitted only when the environment contains a symmetric pointer and a matching consumed ordering exists for the pointer variable, rank, and address.

3.7 Trace semantics

A trace is a list-indexed inductive relation generated by single-step transitions. The semantics distinguishes an action sequence from a mere collection of state facts, allowing provenance to be stated over the complete trace. The calculus models no state-changing data-store transition. It represents notification state and abstract data-pointer metadata in a simplified address namespace.

3.8 Operational rule schemas

For clarity, this subsection presents the principal transition constructors in rule form. The notation is descriptive of the Lean constructors; it is not a second semantics. Let $m = (\sigma, \rho, Wit, Pen, Con)$ be a machine, where Wit, Pen, and Con are the witness, pending, and consumed lists. The notification rules are:

$$\frac{r < W}{m \xrightarrow{\mathsf{notifySet}(r,a,v,s)} m[\sigma.S(r,a) := v]}$$

and

$$\frac{r < W}{m \xrightarrow{\mathsf{notifyAdd}(r,a,v,s)} m[\sigma.S(r,a) := \sigma.S(r,a) + v]}.$$

The transition constructors do not check a pointer or operation-tag environment binding: the action already distinguishes set from add and contains the abstract rank, address, and value. Pointer evaluation and source-level operation selection are outside this transition layer. The source-level distinction between a set and an add is retained in the action constructor, because replacing one by the other changes the signal history even when the final counter happens to coincide.

The wait rule is:

$$\frac{n \le S(r,a) \qquad \mathsf{NotificationVisible}(\sigma, r, a, s)}{m \xrightarrow{\mathsf{wait}(r,a,n,t,s,o)} m[Wit := w :: Wit,\ Pen := p :: Pen]}.$$

Unlike expression-level wait evaluation, this transition rule does not impose a positive-count premise. Here $w = (r, a, n, s, o)$ and $p = (t, r, a, n, s, o)$. The rule records both the ordering witness and the token variable. The two records serve different purposes: the witness represents the successful synchronization event, while the pending record is the resource that a later consume must spend.

The consume rule can be written as:

$$\frac{\begin{array}{c}\rho(x) = \mathsf{ptr}(\tau, r, a, \mathsf{true}) \qquad \rho(t) = \mathsf{token}(a', n, s, o) \qquad a' = a \\ \exists p \in Pen.\ p.\mathsf{tokenVar} = t \wedge p.\mathsf{signalRank} = r \wedge p.\mathsf{signalAddr} = a \\ \wedge\ p.\mathsf{scope} = s \wedge p.\mathsf{ordering} = o\end{array}}{m \xrightarrow{\mathsf{consume}(x,t)} m[Pen := \mathsf{filter}_t(Pen),\ Con := c(x, t, a, r) :: Con]}.$$

The notation filter_t denotes the list operation used by the Lean definition to remove pending records for the consumed token variable. The pending-record match does not compare its expected value with the token's observed value. The consumed record stores only the value variable, token variable, signal address, and rank; scope and ordering are checked against the pending record at consume time, but are not retained by the consumed record or matched by the later load. This asymmetry is a conscious abstraction and is discussed in the limitations section.

Finally, load is a permission transition:

$$\frac{\rho(x) = \mathsf{ptr}(\tau, r, a, \mathsf{true}) \quad c(x, t, a, r) \in Con}{m \xrightarrow{\mathsf{load}(x)} m}.$$

The rule is a self-loop because the expression evaluator, rather than the transition relation, returns the memory value. The self-loop still matters: it permits the theorem to quantify over a final action and to infer its history from the consumed capability required by the premise.

3.9 Capability records and ownership

There are three related notions of identity in the model. A signal location is identified by rank and address. A token is identified by its environment variable and runtime metadata. A consumed ordering is identified by the value variable, token variable, signal address, and rank. These identities should not be conflated. For example, two token variables may contain equal runtime tokens, while only one may have a pending capability in the machine.

The ownership map is used by `NotificationVisible`. Its role is to prevent a malformed signal location from being treated as a visible event: the owner must be below the world size. It does not encode a distributed ownership protocol, and it does not require the current rank

to equal the owner. Thus the model can express a remote signal observation while retaining a basic well-formedness check.

The world size occurs in several premises rather than in one global invariant. Pointer values require their owner to be in range; symmetric addressing requires its target to be in range; notification transitions require the signal rank to be in range; and visibility requires the signal owner to be in range. This repetition is useful in the Lean proofs because each constructor exposes the exact arithmetic fact it needs. It also identifies a future refactoring opportunity: a dependent location type could package these bounds, but would make the current examples and theorem statements less direct.

3.10 Invariant decomposition

The main safety argument is the conjunction of four smaller facts. First, *world preservation* says that transitions do not change the state's world size. Second, *environment preservation* says that the transition rules inspect but do not mutate the environment. Third, *capability origin preservation* says that every pending and consumed record has a history origin. Fourth, *load admissibility* says that a load can occur only when its pointer has a matching consumed record.

The first two facts support ordinary typing and are independent of the wait protocol. The third fact is generated by wait and consume. The fourth fact is the final interface between the protocol and data access. Separating them prevents a common proof error: using a well-typed pointer as if it already carried synchronization permission. In this calculus, pointer typing proves shape and rank bounds; consumed capability proves history.

The decomposition also explains why the initial-state hypothesis matters. If the initial machine were allowed to contain arbitrary pending or consumed records, the origin theorem would need an explicit origin history for those records. The universal load theorem instead assumes both lists are empty, so all capabilities used by a load must have been generated by the trace itself.

3.11 Blocked evaluation versus absent transitions

The expression relation includes `blocked` as a result because a wait may be evaluated before its signal reaches the expected value. A blocked result is not a transition: it does not add a witness, does not add a pending token, and does not advance a machine trace. This distinction is reflected in the Lean development by separate constructors for `waitBlocked` and the successful wait transition.

The distinction is particularly important for progress claims. From a blocked evaluation one can conclude that the observed signal is below the expected value, but one cannot conclude that the signal will eventually increase. From a successful evaluation one can extract visibility and a token, but one cannot conclude that a consumer will execute consume. The artifact therefore proves a conditional local progress characterization rather than a liveness theorem.

3.12 Relationship between expression and transition premises

Expression evaluation and transition execution share the state and environment vocabulary but are not definitionally identical. A successful expression wait contains enough information to construct the token returned to the caller. The transition wait additionally records that success in the machine's capability lists. Similarly, expression-level consume checks runtime token metadata, whereas transition-level consume checks pending provenance.

Table 1 Selected theorem contracts in the Lean artifact.

Theorem	Contract
Eval.1	One expression has one result in a fixed state and environment.
Eval.2	A typed variable returns a value of the context type.
Eval.3	A successful wait returns a scalar token.
Eval.4	Symmetric addressing preserves type and bounds.
Eval.5	A successful load is covered by store typing.
Step.1	A successful wait exposes visibility.
Step.2	Notification changes only its signal cell.
Step.3	Wait adds a witness and pending token.
Step.4	Consume has a matching pending capability.
Step.5	Load has a matching consumed ordering.
Trace.1	Capabilities have wait/consume origins.
Trace.2	A permitted load follows matching wait and consume.

This two-level design permits negative examples that are useful for auditing. An environment can contain a correctly shaped token and therefore satisfy the expression consume rule, while no transition consume is possible because the pending list is empty. Conversely, a pending capability alone is insufficient for consume if the environment token or pointer metadata does not match. The formalization thus distinguishes shape errors, readiness errors, and provenance errors rather than collapsing them into one failure result.

4 Mechanized Results

The results are organized into three layers. Core-level lemmas reason about values produced by one expression. Step-level lemmas reason about the shape of one enabled action. Trace-level lemmas lift the step facts to arbitrary finite executions. This organization mirrors the Lean files and avoids hiding a transition premise inside an informal operational argument.

4.1 Theorem contracts

Table 1 summarizes the principal theorem contracts. A theorem whose conclusion mentions typing concerns only the encoded types and values; a theorem mentioning origins concerns the explicit lists in `Machine`. In particular, the table makes visible that provenance and memory visibility are different obligations.

4.2 Capability-origin invariant

The theorem `trace_preserves_capability_origins` states that if each pending and consumed capability in the initial machine has an origin in a history prefix, then after any admitted trace, every resulting capability has a corresponding origin in the concatenated history. A pending token's origin is a matching wait action. A consumed ordering's origin is a wait followed later by a matching consume.

The proof proceeds by induction on the trace derivation. Notification and load steps preserve the capability lists. A wait introduces one pending capability whose origin is the current action. A consume removes matching pending entries and introduces one consumed capability; the transition premise supplies the pending token whose wait origin extends to a wait–consume decomposition. Existing origins are extended by appending the current action.

4.3 Universal load provenance

The central theorem, `load_has_prior_wait_and_consume`, assumes empty initial pending and consumed lists, a trace to a final machine, and a permitted load step from that machine. It concludes that there exist lists $before$, $middle$, and $after$, along with metadata, such that

$$actions = before +\!+ [wait] +\!+ middle +\!+ [consume] +\!+ after,$$

and the loaded environment variable has the corresponding symmetric pointer. The consume uses the same token variable and value variable as the loaded pointer; the consumed record also matches that pointer's rank and address. The theorem does not show that the wait's expected value, scope, or ordering matches the token metadata or the later load: those fields are not retained in the consumed record. Thus the result rules out, within the calculus, a permitted load lacking a preceding wait and consume in the trace. and the loaded environment variable has the corresponding symmetric pointer. The consume uses the same token variable and value variable as that pointer, and the consumed record matches its rank and address. The theorem does not show that the wait's expected value, scope, or ordering matches the token metadata or the later load: these fields are not retained in the consumed record. Thus the result rules out, within the calculus, a permitted load lacking a preceding wait and consume in the trace.

The theorem does not show that the wait made real memory visible or that the compiler retained an SSA dependency. The consume transition's capability is an abstract machine invariant. The theorem also does not establish program termination or guarantee that a wait eventually succeeds.

4.4 Type and progress results

The core includes preservation statements for typed environments and stores across modeled steps and traces, along with conditional wait progress. The latter says that, under the visibility premise, either the wait transition is enabled or the observed signal is below the expected value. This is a local characterization of the modeled wait; it is not deadlock freedom, fairness, or collective progress.

4.5 Proof structure and induction measures

The expression lemmas use inversion on the relevant `Eval` constructor. For example, inversion on a successful symmetric-addressing derivation exposes the pointer lookup and target-bound premise, so the conclusion follows by the definition of `ValueHasType`. The store lemma is similarly a direct application of the quantified premise in `StoreWellTyped`.

The transition lemmas use the concrete list operations in the machine. A wait step prepends one pending record and one witness; a consume step filters the pending list and prepends one consumed record. The key proof obligation is therefore not list membership alone but preservation of the metadata relation between the environment and the capability record. The implementation proves this by destructing the matching pending record supplied by the consume rule.

The trace theorems use induction on the derivation rather than induction on an untrusted action list. The empty case establishes the initial capability origin assumption. In the step case, the induction hypothesis handles all pre-existing capabilities, while the action-specific lemma handles the one new capability introduced by wait or consume. The load theorem then applies the origin invariant to the consumed record required by the final load step and decomposes the action list into prefix, wait, middle, consume, and suffix.

This proof architecture is intentionally auditable. It makes the source of each premise visible in Lean and prevents a theorem about arbitrary lists from being mistaken for a theorem about enabled executions.

5 Examples and Source Validation

The Lean examples construct a producer state with a signal value and a consumer trace of `wait -> consume -> load`, starting with empty capability lists. Negative results show that a load is not derivable from a machine lacking a consumed capability, and that an environment token alone cannot authorize consume when no pending wait capability exists. These examples demonstrate the transition rules, while the universal theorem supplies the general trace property.

5.1 Concrete positive trace

The positive example uses a two-rank world. The consumer is rank zero and the signal owner is rank one; both the signal and data address are seven. The signal owner is rank one and its counter is initialized to one. The environment contains a symmetric i32 pointer at variable one, a token variable two, and the corresponding signal pointer and scalar arguments used by the wait. The initial pending and consumed lists are empty.

The trace starts from `emptyConsumerMachine`, whose signal is already initialized to one; it does not include a producer notification or data-store transition. The first action is a wait for expected value one. Its enabledness follows from $1 \leq S(1, 7)$ and `NotificationVisible`. The resulting machine contains a pending record connecting token variable two to rank one and signal address seven. The second action consumes variable one with token two. The address and scope/order fields agree with the pending record, so the pending record is removed and `ConsumedOrdering` records the pointer variable, token variable, signal address, and rank. The third action loads variable one. It is enabled because the consumed record matches the pointer's rank and data address. This is the concrete witness used by the universal load theorem.

5.2 Negative traces and blocked waits

The first negative example starts with the same environment but an empty consumed list and attempts a load. The load premise cannot be constructed, because `load` requires a matching consumed ordering. The second starts with a token value in the environment but no pending wait capability. Although the token is well typed and expression-level consume can return its value, the transition-level consume premise fails. This illustrates why the scalar token type is weaker than provenance.

A third classification sets the signal below the expected value. The expression evaluator derives `waitBlocked`, and no successful wait transition is available. This is not a safety violation: the model distinguishes absence of readiness from an invalid token or an unauthorized load. The conditional progress theorem captures exactly this distinction.

5.3 What the examples do not test

The examples are proof-producing unit cases, not distributed executions. They do not test network latency, multiple concurrent producers, signal reuse across epochs, allocation failures, compiler scheduling, or device memory behavior. They also do not establish that the source

tutorial necessarily lowers to the same action sequence. Their role is to make the abstract premises and the counterfactual failures executable at the Lean level.

A separate Python validator parses selected upstream tutorial files using the standard-library AST module. It checks three source patterns: a notify/wait producer–consumer example, an intra-node all-gather, and an overlapping all-gather/GEMM example. Mutation checks reject selected in-memory source alterations, including removal of token consumption or signal-bearing communication. This validator checks names and relative source-call shapes. It is not an execution test, a translation validator, or evidence about generated IR or hardware behavior.

5.4 Artifact and reproducibility

The formal artifact is a Lake project pinned to Lean 4.19.0. Building the project with `lake build` checks the core, transition semantics, and examples together. The source audit is run with Python 3 and an explicit local source root; pass the path to the audited checkout explicitly with `python3 validation/validate_source_patterns.py -source-root PATH/TO/Triton-distributed-ascend -mutate`. The validator reports 12 positive source-pattern checks and, with `-mutate`, three additional negative mutation checks. The validator uses only the Python standard library. The revision string is reported for provenance but is not checked against Git; reproduction requires checking out revision `1e6d688` first.

The Lean source contains no `sorry`, `admit`, or user-declared `axiom`. This statement concerns the checked artifact, not explicit theorem premises such as `NotificationVisible`; these assumptions are deliberately exposed in theorem statements. The paper's PDF is built using the official `lipics-v2021` class and the `plainurl` bibliography style.

6 Relationship to the Implementation

6.1 Correspondence obligations

The relationship between the source API and the calculus can be stated as a sequence of obligations. The first is syntactic: a source call must have the argument shape recognized by the audit. The second is typing: translated expressions must inhabit the corresponding `HasType` judgments. The third is enabling: the source execution must supply the signal and visibility premises required by a transition. The fourth is provenance: the token consumed by a data value must be the token generated by the matching wait. The fifth is refinement: source and target executions must agree on visibility and dependency preservation.

Only the middle part of this chain is present here. Lean defines what counts as a typed and enabled core execution, while the Python validator checks a small set of source shapes. There is no function from Python ASTs, Triton IR, or MLIR values to `Expr` values. In particular, a validator success does not construct an `Env`, `Ctx`, or `Machine` and cannot be used as a premise of a Lean theorem without an additional translation proof.

An eventual translation would need to specify how source objects become abstract addresses and variables. It would also need to state whether a source token is linear, affine, or freely copyable. The current model records pending capability consumption but does not encode uniqueness of a runtime token in the environment. A source-level proof could enforce linear use or prove that duplicated token values cannot create an additional consumed ordering. This is a design choice rather than an implementation detail.

6.2 Compiler transformations

The abstract consume action represents a dependency edge between communication completion and a later data use. A compiler may preserve that edge by keeping a data dependency, attaching an ordering attribute, or lowering to a backend primitive with an equivalent effect. The calculus does not distinguish these strategies. It records the edge as a capability and proves that a load is guarded by it.

This abstraction does not prove optimizer preservation. Common-subexpression elimination could duplicate or merge token-producing expressions; dead-code elimination could remove a consume whose result appears unused; scheduling could move a load before communication; and lowering could erase scope or ordering metadata. A compiler correctness theorem would need an observation relation that treats the capability edge as semantically relevant. It would then show that every target load corresponding to a source load has a source wait and consume, or that a target primitive provides an equivalent guarantee.

The action-list theorem also does not assume syntactic adjacency. Its middle list can contain only supported actions (notify, wait, consume, or load), not arbitrary computation. The required fact is the existence and order of the capability edge, not that the three operations are consecutive.

6.3 Signal reuse and epochs

Signals are natural-number counters, and waits compare an expected value with the current counter. This supports monotone protocols such as waiting for one and later two, but it does not establish that a value belongs to the intended communication epoch. If a signal address is reused, an old value may satisfy a later wait unless allocation and epoch discipline are supplied externally.

The witness list records the expected value and can distinguish waits for different counter values. The consumed ordering currently stores only the pointer variable, token variable, signal address, and rank. It does not retain the expected value. Consequently, the load theorem proves a matching wait/consume history for the recorded association, but not a global theorem that a load observes the newest producer epoch. Adding expected value and scope/order fields to the consumed record would support that strengthening.

6.4 Alternative modeling choices

One alternative would encode protocol state in dependent types. A token type could carry its signal address and expected value, and consume could require a proof that the token was produced by wait. This would make invalid consume expressions unrepresentable, but would move the central result from trace reasoning into type formation. The present model keeps tokens scalar-typed and proves provenance as an invariant over enabled transitions, matching the fact that compiler IR values often carry less static information than a protocol requires.

A second alternative would model complete configurations with control-flow locations and multiple ranks. That would support interleaving and scheduling theorems, but would add machinery not needed for the local wait–consume–load property. The current machine represents one abstract protocol history and leaves interleaving to the supplied trace. A third alternative would expand `NotificationVisible` into a weak-memory model. Keeping it as an interface lets a backend-specific happens-before or message-passing relation replace the predicate while preserving the provenance proof's structure.

The model is informed by source operation signatures but abstracts several API dimensions. Rank and world-size queries are represented at a high level; pointer/count evaluation for

waits is abstracted; the wait is represented by a scalar rank/address/expected-value tuple; and arbitrary SSA token dataflow is not modeled. The source operation `consume_token` accepts a value and token independently, while the calculus adds explicit matching premises. Ordinary loads are guarded by a capability in the calculus, whereas the Python-level load API does not generally reject a load lacking such a consume dependency.

Accordingly, the relation is a deliberate strengthening and abstraction for proof tractability, not an equivalence. The implementation may express programs not admitted by the calculus. Conversely, the calculus has no translation theorem proving that the source programs accepted by the validator map to its actions. Potential mismatch cases include unrelated tokens, signal/data pointer mismatch, omitted consume, unsafe signal reuse, rank or allocation disagreement, and insufficient memory visibility. These are identified as obligations and possible failure modes, not as experimentally confirmed bugs in a particular kernel.

7 Assumptions and Limitations

The results rely on explicit premises and omitted mechanisms:

- `NotificationVisible` is assumed for successful waits; no CANN, CUDA, NVSHMEM, or ROCSHMEM rule is proved.
- Compiler preservation of the consume dependency through optimization and lowering is not established.
- Symmetric allocation agreement, signal initialization and epoch discipline, and valid collective participation are not modeled as global invariants.
- The model has no source/MLIR-to-core translation or simulation, no backend refinement, and no hardware execution semantics.
- The core does not prove deadlock freedom, fairness, buffer lifetime, general data-race freedom, or end-to-end kernel correctness.

A stronger result would require a typed source or IR subset, a translation into the core, a simulation theorem, and evidence or a formal refinement connecting visibility and dependency preservation to each supported backend. Targeted litmus tests and signal-reuse tests would complement, but not replace, those proofs.

8 Related Work

Distributed accelerator programming. Triton-Distributed provides a programming model for distributed GPU kernels, including communication and synchronization operations intended to overlap communication with computation [12]. TileLink studies tile-centric primitives and optimization techniques for generating such overlapping kernels [13]. These works address the programming model and performance of distributed accelerator computations; our goal is different. We isolate one wait–consume–load discipline and formalize the trace evidence required by an abstract machine before it permits a load. The calculus does not model the complete Triton-Distributed language, reproduce its compiler, or establish the performance or correctness of its kernels.

Choreographic programming and session types. Choreographic programming describes interactions from a global perspective: a protocol specifies the roles and communication events of a distributed computation, and endpoint programs are obtained by projecting that description. Multiparty session types provide related techniques for specifying structured conversations and checking that participants respect their communication protocols [5]. Such

approaches typically reason about the consistency of multiple participants and, in some settings, communication safety or progress.

The present calculus is adjacent to this line of work, but it is not a choreography or a multiparty session-type system. It has no global protocol syntax, endpoint projection, or collection of independently executing rank-indexed processes. Its trace is an abstract sequence of enabled actions, not a theorem that projected endpoints implement a global specification. The connection is instead at the protocol level: ranks and signal metadata identify participants and communication resources, while pending and consumed capabilities record local obligations induced by synchronization. A choreography-aware extension could define a global producer–consumer protocol, project it to rank-indexed actions, and prove that the projected traces are admitted by this calculus. The current provenance theorem could then serve as a local obligation in a larger projection or refinement proof.

Session-typed and separation-logic reasoning. Session-typed reasoning in Actris combines protocol specifications with separation logic, enabling modular reasoning about communication and shared resources [4]. Iris provides a general foundation for higher-order concurrent separation logic [6], while Aneris and Trillium develop mechanized reasoning for distributed systems and intensional refinement [8, 10]. These frameworks have substantially richer accounts of concurrency, resources, and composition than the transition system studied here. Our pending and consumed records are protocol resources in a narrower sense: they record the origin and use of a synchronization capability, but are not assertions in a separation logic and do not support modular reasoning about arbitrary concurrent components. The artifact therefore complements rather than implements these logical frameworks. General work on logical type soundness provides a broader account of the relationship between typing and semantic safety [11]; our preservation results concern only the specific runtime predicates of the encoded calculus.

Weak memory and synchronization semantics. A synchronization protocol is meaningful only relative to an account of ordering and visibility. Work on release–acquire reasoning and weak-memory fences studies how synchronization operations constrain observable executions [7, 1]. In our model, `NotificationVisible` is an explicit premise of a successful wait. The calculus proves that a permitted load has the required abstract wait–consume history; it does not derive visibility from a weak-memory semantics, nor does it establish that a particular CANN, CUDA, NVSHMEM, or ROCSHMEM operation implements that premise. This separation makes the backend obligation visible, but leaves its justification to a future target-specific semantics or refinement proof.

GPU verification, hardware models, and compiler correctness. GPUVerify checks classes of data races and barrier-divergence errors in GPU kernels [3]. Kami demonstrates mechanized hardware specification and modular verification [2], while CompCert proves semantic preservation for a defined compiler pipeline [9]. These works address different layers and broader properties than our abstract protocol theorem. In particular, we do not prove race freedom, hardware refinement, or preservation of synchronization dependencies through compiler optimization and lowering. The capability trace provides a candidate intermediate-level property for such a proof: a compiler account could relate source or IR dependencies to the abstract consume capability and then show that the target implementation preserves the corresponding ordering.

Positioning. The contribution is thus narrower than choreographic programming, session typing, weak-memory verification, GPU-kernel verification, or compiler verification. It is a mechanized local protocol model with a provenance invariant: from empty initial capability state, every load admitted by the abstract transition relation has an earlier matching wait and consume. Connecting this result to global protocol specifications or executable Triton-Distributed kernels requires additional endpoint projection, source-to-core translation, compiler-preservation, and backend-refinement results. The comparison is focused rather than systematic and does not establish novelty or priority relative to all related work.

9 Conclusion

We presented a Lean core calculus for a wait–consume–load synchronization discipline motivated by Triton-Distributed. The mechanized provenance invariant and universal load theorem show that the transition system itself requires a matching earlier wait and consume before it permits a load. A source-pattern audit checks selected examples and exposes, rather than closes, the gap between source operations and the abstract calculus. The immediate technical value is a precise statement of the obligations a future source-to-core and backend-refinement account must discharge. Establishing such a connection, and validating target-specific memory-ordering assumptions, remains future work.

Data Availability. The accompanying source tree contains the Lean source, validation script, source audit, and build instructions. The artifact is built with Lean 4.19.0 and Lake; source-pattern validation uses Python 3 and the standard library. The audited upstream revision is recorded as `1e6d688`; the audit reads checkout working files and does not verify Git history.

References

1. Jade Alglave, Luc Maranget, Susmit Sarkar, and Peter Sewell. Fences in weak memory models. In *Computer Aided Verification*, volume 6174 of *Lecture Notes in Computer Science*, pages 258–272, 2010. doi:10.1007/978-3-642-14295-6_24.
2. Adam Chlipala. Kami: A platform for high-level parametric hardware specification and its modular verification. In *Proceedings of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 1–14, 2017. doi:10.1145/3062341.3062350.
3. Alastair F. Donaldson, Darren Liew, Cedric Fumex, et al. Gpuverify: A verification framework for gpu kernels. In *Proceedings of the 2012 ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications*, pages 113–132, 2012. doi:10.1145/2384616.2384654.
4. Jonas Kastberg Hinrichsen, Jesper Bengtson, and Robbert Krebbers. Actris: Session-type based reasoning in separation logic. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 672–687, 2020. doi:10.1145/3385412.3385970.
5. Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. *Journal of the ACM*, 63(1):9:1–9:67, 2016. doi:10.1145/2827695.
6. Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Bart Jacobs, Jean Pichon-Pharabod, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018. doi:10.1017/S0956796818000151.
7. Jan-Oliver Kaiser, Hoang-Hai Dang, Derek Dreyer, Ori Lahav, and Viktor Vafeiadis. Reasoning under release-acquire consistency. *Electronic Proceedings in Theoretical Computer Science*, 263:1–15, 2017. doi:10.4204/EPTCS.263.3.

8 Morten Krogh-Jespersen, Amin Timany, Marit Edna Ohlenbusch, Simon Oddershede Gregersen, and Lars Birkedal. Aneris: A mechanised logic for modular reasoning about distributed systems. In *Programming Languages and Systems*, volume 12075 of *Lecture Notes in Computer Science*, pages 336–365, 2020. doi:10.1007/978-3-030-44914-8_13.

9 Xavier Leroy. Formal verification of a realistic compiler. *Communications of the ACM*, 52(7):107–115, 2009. doi:10.1145/1538788.1538814.

10 Amin Timany, Simon Oddershede Gregersen, Leo Stefanesco, Jonas Kastberg Hinrichsen, Leon Gondelman, Abel Nieto, and Lars Birkedal. Trillium: Higher-order concurrent and distributed separation logic for intensional refinement. *Proceedings of the ACM on Programming Languages*, 8(POPL):241–272, 2024. doi:10.1145/3632851.

11 Amin Timany, Robbert Krebbers, Derek Dreyer, and Lars Birkedal. A logical approach to type soundness. *Journal of the ACM*, 71(6):Article 40, 2024. doi:10.1145/3676954.

12 Size Zheng, Wenlei Bao, Qi Hou, Xuegui Zheng, Jin Fang, Chenhui Huang, Tianqi Li, Haojie Duanmu, Renze Chen, et al. Triton-distributed: Programming overlapping kernels on distributed ai systems with the triton compiler, 2025. URL: https://arxiv.org/abs/2504.19442, arXiv:2504.19442.

13 Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, et al. Tilelink: Generating efficient compute-communication overlapping kernels using tile-centric primitives, 2025. URL: https://arxiv.org/abs/2503.20313, arXiv:2503.20313.